

GLOBAL ACADEMY OF TECHNOLOGY

growing ahead of time

Autonomous Institute, Affiliated to VTU

Department of Artificial Intelligence & Machine Learning

AML23702

Advance Natural Language Processing

AML23702_Syllabus.pdf

VASUGI I

ASSISTANT PROFESSOR

DEPARTMENT OF AI & ML

GLOBAL ACADEMY OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

Module-2
Question Answering and Chatbot

Question Answering and Chatbot : Information Retrieval, IR-based Factoid Question answering, Knowledge-based Question Answering, Classic QA Models, Using multiple information sources: IBM's Watson, Evaluation of Factoid Answers, Properties of Human Conversation, GUS: Simple Frame-based Dialogue Systems, Dialogue System Design.

Textbook : Speech and Language processing: An introduction to Natural Language Processing, Computational Linguistics and speech Recognition by Daniel Jurafsky and James H. Martin

Textbook link (ch 25 & 26) - ed3book-3rd edition.pdf

GLOBAL ACADEMY OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

Pre-Session Reading

- An overview of question answering systems, how they differ from chatbots, and the role of information retrieval in finding answer-bearing passages - https://www.sciencedirect.com/science/article/pii/S0952197625009972
- A short reading on factoid QA pipelines: question analysis, document retrieval, passage ranking, and answer extraction - https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9622709

Information Retrieval

IR systems retrieve relevant documents from a large corpus based on a query, using keyword matching, TF-IDF, BM25, or semantic search.

How It Works

1. Query is parsed and preprocessed
2. Index is searched (inverted index / embeddings)
3. Documents ranked by relevance score (BM25, cosine sim)
4. Top-k documents returned as answers

Tech Stack

- Elasticsearch / OpenSearch
- Apache Solr
- BM25 (rank_bm25 Python)
- FAISS / Annoy (vector search)
- Lucene, Whoosh

Real-World Use Cases

- Google/Bing web search
- Legal document retrieval
- Healthcare literature search (PubMed)

Question Answering - Overview

- QA systems: retrieve or generate answers to user questions
- Two major paradigms: **IR-based QA** and **Knowledge-based QA**
- Modern QA integrates multiple information sources (text, structured data, web)
- Applications: search engines, assistants, customer support, chatbots
- **Types of Questions**
  - **Factoid questions**: seek short, factual answers (names, dates, numbers)
  - **List questions**: require multiple answers
  - **Descriptive questions**: need longer, explanatory responses
- QA systems often specialize in factoid questions due to their structured nature

IR-based Factoid Question Answering

- Treats QA as an **information retrieval** problem
- Goal: return short factual answers (names, dates, numbers)
- Uses large text corpora (web, news, Wikipedia)
- Pipeline: **question → query → retrieve passages → extract answer**
- Why IR-based works?
  - Many factoid answers appear explicitly in text
  - Web-scale corpora increase likelihood of finding exact answers
  - Retrieval models (BM25, TF-IDF, embeddings) help rank relevant passages
  - Extraction models identify answer spans within retrieved text

IR-based factoid QA – Three stages

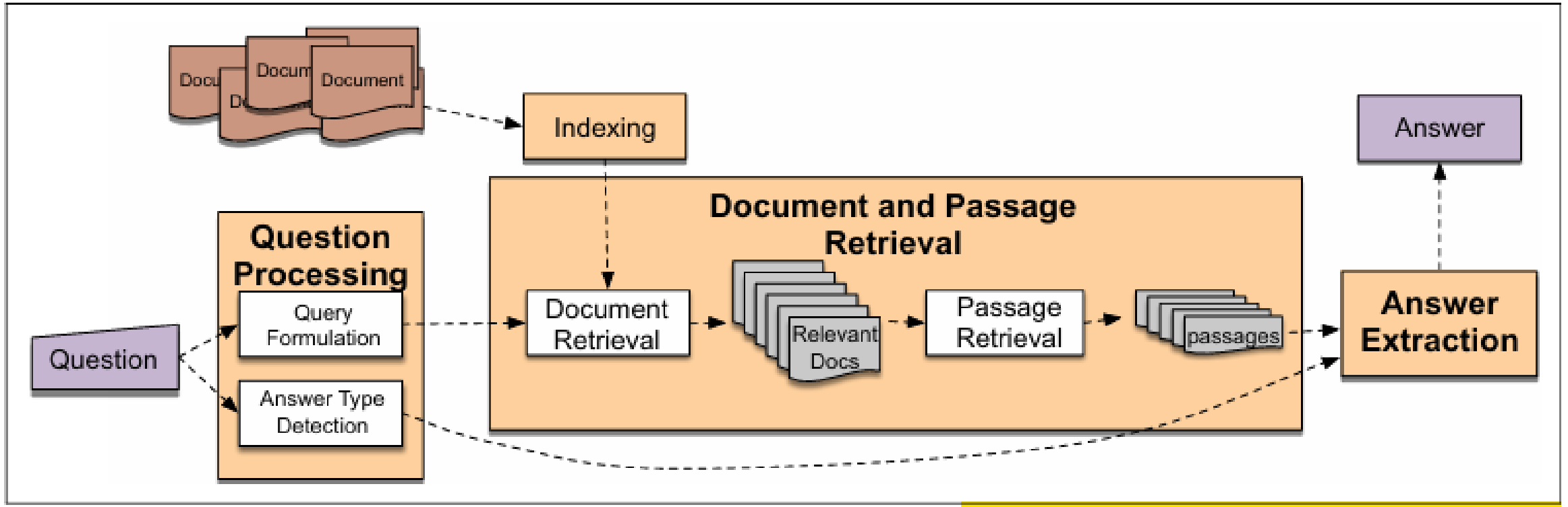


Figure 25.2 IR-based factoid question answering has three stages: question processing, passage retrieval, and answer processing.

IR-based factoid QA - Query Processing

Question processing extracts four things from a question

Query	Answer type	Focus	Question type
The keywords passed to the IR system to match documents	The entity type of the answer (person, location, time…)	The words likely to be replaced by the answer	Definition, math, list…?

Example

Which US state capital has the largest population?



query	answer type	focus
"US state capital has the largest population"	city	state capital

Query Formulation

- Query formulation is the process of turning a natural language question into a search query that an information retrieval (IR) system can use to fetch relevant documents.
- Question → Query → IR Engine → Candidate Passages → Answer Extraction
- Task: create a **query (list of tokens)** to send to an IR system
- Goal: retrieve documents likely to contain answer strings

Query Formulation : Query Expansion

- Query expansion adds extra terms to the query to increase the chance of matching the answer.
- Why needed?
  - In small corpora, answers appear in **only one form**.
  - That form may **not match** the question wording.
- Types of expansions:

  a. Morphological variants

  - Add inflected forms: *invent, invented, invention, inventing*

  b. Synonyms

  - Add semantic equivalents: *created, developed, originated*
- This increases recall by covering more possible answer phrasings.

Query Formulation

Web QA

Use the **entire question** minus the wh-word

"When was the laser invented?"



"laser invented"

Usually enough for a web search engine to find the right page

Smaller corpora (Wikipedia, enterprise docs)

1. Index documents as tf-idf vectors
2. Unigrams alone may not be enough
3. Use bigrams: "Valley of", "Kings is"
4. Compare query vs. document by cosine similarity

Why bigrams?

- Answers are often multiword
- More context than single words
- Less ambiguity: "Valley of the Kings"

Question Classification : Answer Types

Question classification = finding the **answer type** — the named-entity category of the answer

"Who founded United Airlines?"

→ **PERSON**

"What Canadian city has the largest population?"

→ **CITY**

Knowing the answer type lets the system

Focus
Focus only on relevant sentences

Prioritize
Prioritize specific named entities

Reduce
Reduce unnecessary computation

Example: PERSON questions → examine sentences containing people names

Question Classification Approaches

1. Rule-Based

- Hand-written patterns
- Example: detecting BIOGRAPHY using syntactic templates
  - who {is | was | are | were} PERSON
- High precision, limited coverage

2. Supervised Learning

- Train classifiers on labeled question datasets
- Methods:
  - Feature-based ML
  - Neural models (CNNs, LSTMs, transformers)

Question Classification Approaches

- Most question classifiers, are based on supervised learning, trained on databases of questions that have been hand-labeled with an answer type (Li and Roth, 2002).

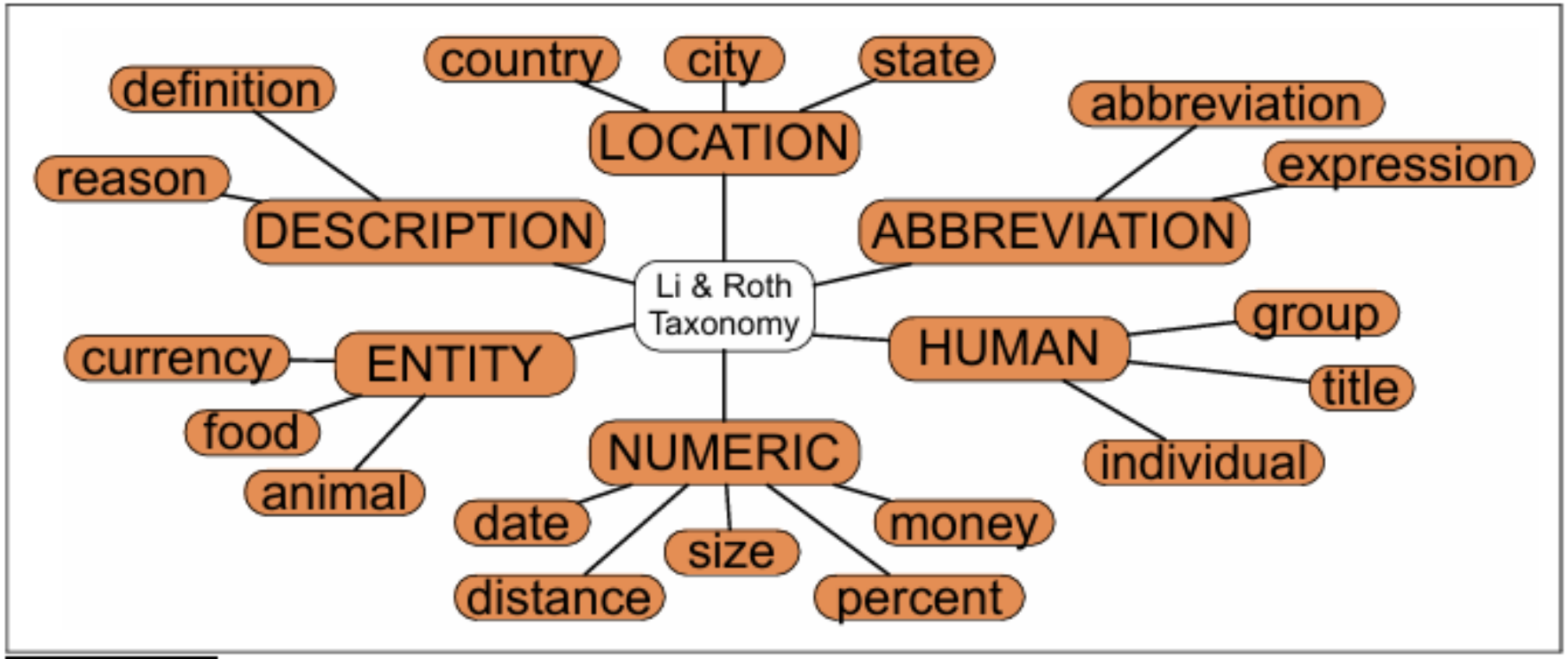
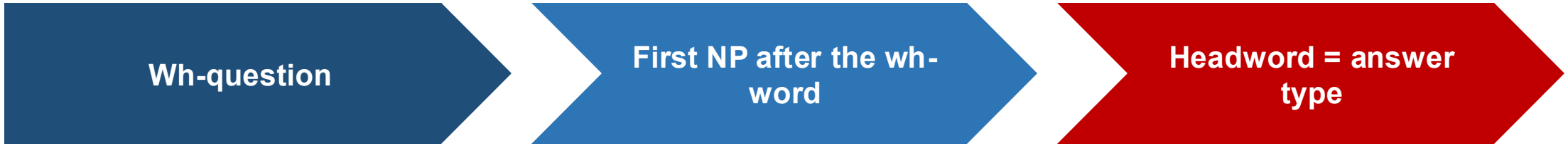


Figure 25.3 A subset of the Li and Roth (2005) answer types.

Answer Type Word (Question Headword)



Which **city** in China has the most foreign financial companies?

city → CITY / LOCATION

What is the state **flower** of California?

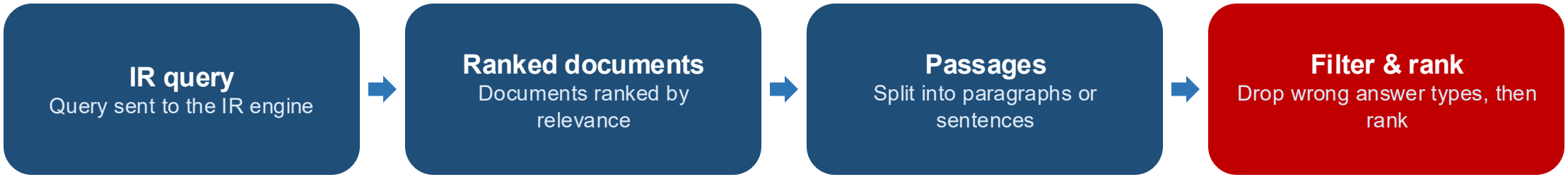
flower → PLANT

The answer type word (question headword) is used as a feature for question classification

Classification Difficulty

- High accuracy for:
  - PERSON
  - LOCATION
  - TIME
- Harder for:
  - REASON
  - DESCRIPTION
  - BIOGRAPHY
- These require deeper semantic understanding

Document and Passage Retrieval



Supervised ranking of the remaining passages uses features like

Answer-type entities	Keyword count	Longest match
Number of named entities of the right type in the passage	Number of question keywords appearing in the passage	Longest exact sequence of question keywords in the passage
Document rank	**Keyword proximity**	**N-gram overlap**
Rank of the document the passage was extracted from	How close the original query keywords are to each other (Pasca 2003; Monz 2004)	N-grams shared between the passage and the question (Brill et al., 2002)

Simplest variant: pass every retrieved passage straight to answer extraction

Answer Extraction

Final QA stage: pull the exact answer span out of a retrieved passage

Span labeling	NER baseline	Where it breaks
Given a passage, identify the span of text that constitutes the answer	Run a named entity tagger and return the span whose type matches the expected answer type	Definition questions have no single named entity type to match

HUMAN | **answer: Manmohan Singh**

"Who is the prime minister of India?"

Manmohan Singh, Prime Minister of India, had told left leaders that the deal would not be renegotiated.

DISTANCE-QUANTITY | **answer: 29,029 feet**

"How tall is Mt. Everest?"

The official height of Mount Everest is 29,029 feet.

Modern systems go further: supervised learning — first feature-based classifiers, then neural span models

Feature-based Answer Extraction

The approach

Train a classifier to decide whether a span or sentence **contains the answer**

Hand-written patterns also help (Pasca, 2003)

such as — **What is autism?**

"…developmental disorders such as autism"

AP , QP — **What is a caldera?**

"the Long Valley caldera, a volcanic crater 19 miles long"

Features used by the classifier

1. **Answer type match —** candidate contains a phrase of the correct answer type
2. **Pattern match —** which hand-written pattern the candidate matched
3. **Matched keywords —** how many question keywords the candidate contains
4. **Keyword distance —** distance between the candidate and the query keywords
5. **Novelty factor —** at least one candidate word is not in the query
6. **Apposition —** candidate is an appositive to a phrase rich in question terms

Neural Answer Extraction

How neural answer extraction works

1. **Encode the question —** Compute a single embedding for the question
2. **Encode the passage —** Compute an embedding for every token in the passage
3. **Score similarity —** Question and answer are semantically close in embedding space
4. **Select the span —** Return the passage span whose embeddings are closest to the question

Intuition: a question and its answer are semantically similar

Reading comprehension: task and data

Origins (Hirschman et al., 1999)
Children's reading tests reused to evaluate machine comprehension

The corpus
120 passages × 5 questions, written for 3rd–6th grade readers

Modern datasets
Built for NLP and large enough to train supervised systems

SQuAD
Wikipedia passages, span answers, plus deliberately unanswerable questions — 150,000+ questions

Beyoncé Giselle Knowles-Carter (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in Houston, Texas, she performed in various singing and dancing competitions as a child, and rose to fame in the late 1990s as lead singer of R&B girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, Dangerously in Love (2003), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy".
Q: "In what city and state did Beyoncé grow up?" A: "Houston, Texas" Q: "What areas did Beyoncé compete in when she was growing up?" A: "singing and dancing"
Q: "When did Beyoncé release Dangerously in Love?" A: "2003"

Figure 25.6 A (Wikipedia) passage from the SQuAD 2.0 dataset (Rajpurkar et al., 2018) with 3 sample questions and the labeled answer spans.

A bi-LSTM-based Reading Comprehension Algorithm

Four steps of the reader

1. **Encode the question** — bi-LSTM over question words, weighted into one vector
2. **Encode the passage** — bi-LSTM over every token of the passage
3. **Compare** — Bilinear attention scores each token against the question
4. **Mark the span** — Two classifiers predict the start and the end token

Goal: find where the answer starts and ends

What each passage token carries

Word embedding — Pretrained vector for the word (GloVe)

Token features — Part of speech and named entity tags

Exact match — Flags words that also appear in the question

Aligned question embedding — Attention links similar but different wordings

Trained end-to-end on datasets such as SQuAD

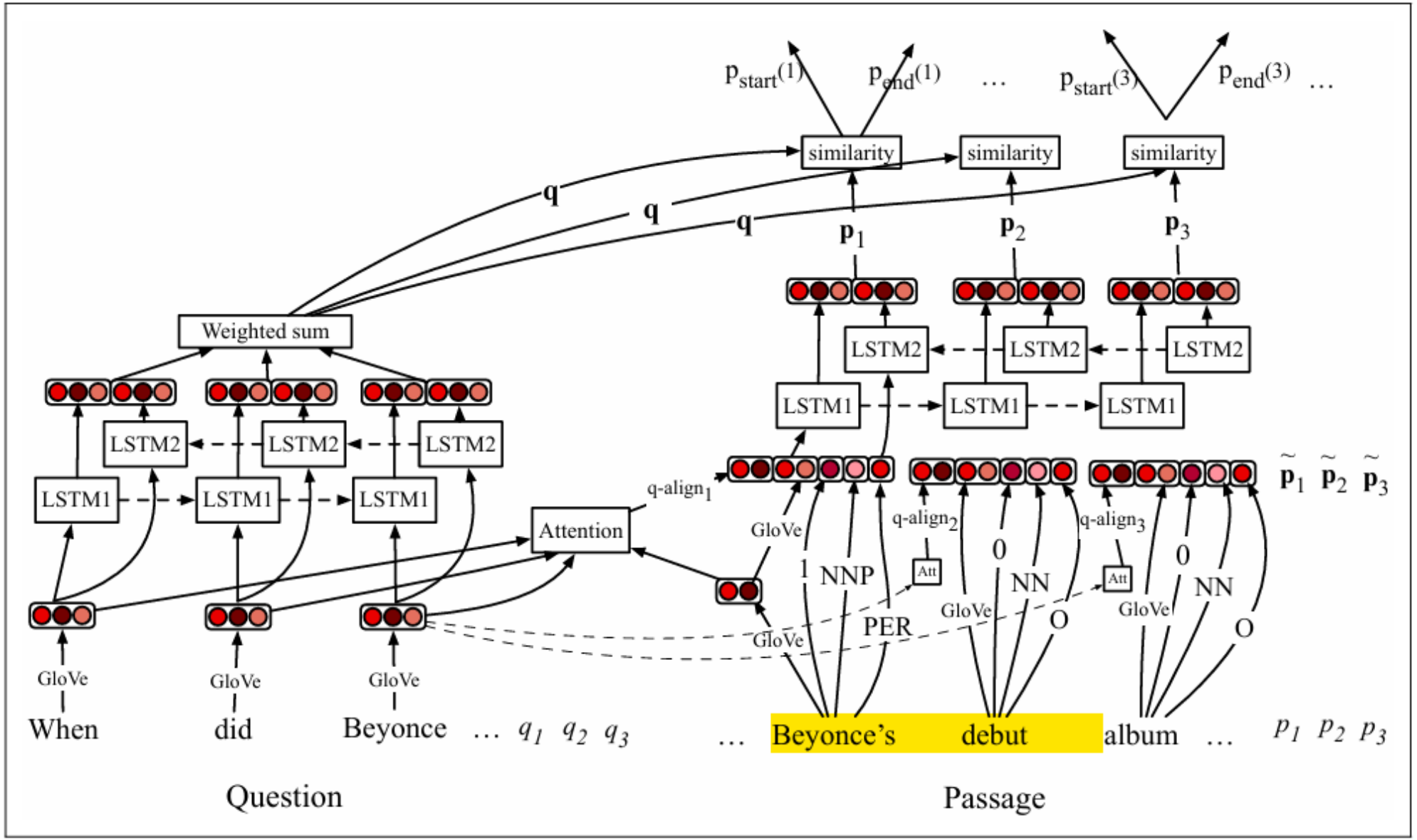


Figure 25.7 The question answering system of Chen et al. (2017), considering part of the question *When did Beyoncé release Dangerously in Love?* and the passage starting *Beyoncé's debut album, Dangerously in Love (2003).*

BERT-based Question Answering

- Contextual embeddings lift QA accuracy above earlier neural readers
- Question and paragraph are fed as one token sequence, split by a [SEP] marker
- BERT emits an embedding for every paragraph token
- Fine-tuning adds two learned vectors: span-start and span-end
- To get a span-start probability for each output token T_i, we compute the dot product between S and T_i and then normalize over all tokens T_i in the paragraph:

$$Pstart_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$$

We do the analogous thing to compute a span-end probability:

$$Pend_i = \frac{e^{E \cdot T_i}}{\sum_j e^{E \cdot T_j}}$$

- The best-scoring valid span becomes the predicted answer

BERT-based Question Answering

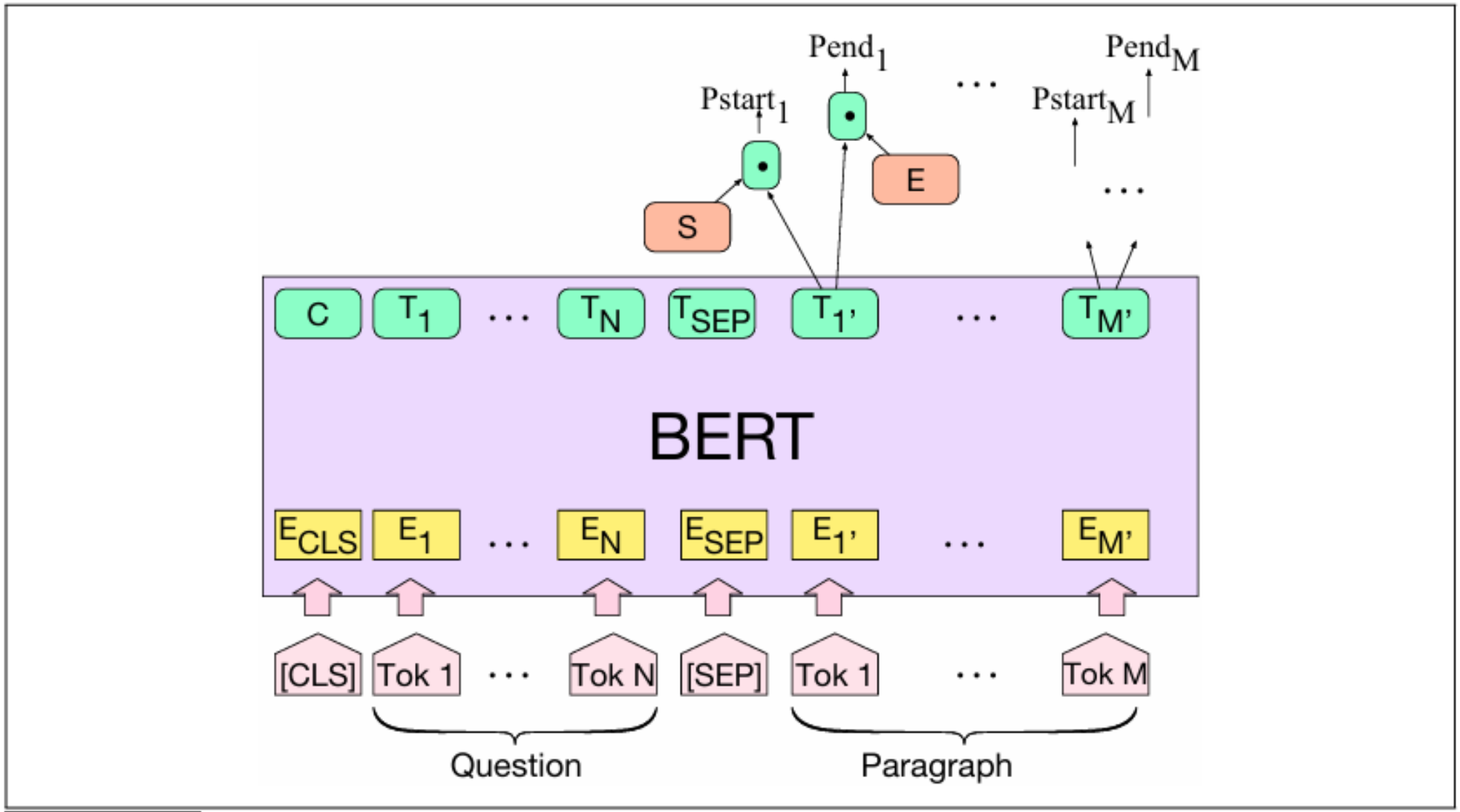


Figure 25.8 The BERT model for span-based question answering from reading-comprehension-based question answering tasks. Figure after Devlin et al. (2019).

Knowledge-based Question Answering

THE CORE IDEA

Answer a natural-language question by mapping it to a query over a structured database.

Text → logical form is done by semantic parsers.

QUERY / KNOWLEDGE FORMS

- SQL, SPARQL, predicate calculus
- Relational DBs or sets of RDF triples
- Freebase, DBpedia — millions of triples from Wikipedia infoboxes

Roots run deep: BASEBALL (Green et al., 1961) answered questions from a baseball database.

AN RDF TRIPLE = PREDICATE + TWO ARGUMENTS

SUBJECT	PREDICATE	OBJECT
Ada Lovelace	birth-year	1815

Factoid QA = ask for the one missing argument in the triple → ?x

"When was Ada Lovelace born?"	**birth-year (Ada Lovelace, ?x)**
"What is the capital of England?"	**capital-city (?x, England)**

The hard part: mapping textual strings like "when was … born" onto canonical knowledge-base relations like birth-year.

Rule-based Methods

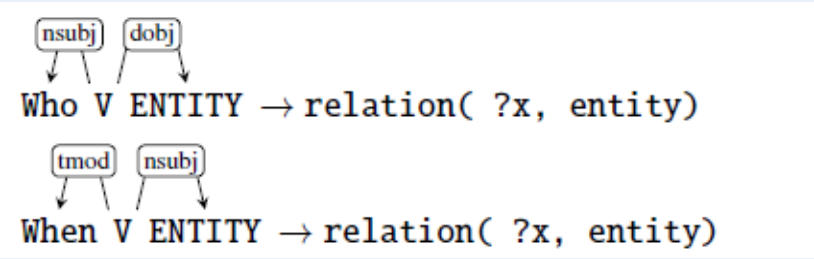
- Handwritten rules pay off for very frequent relations
- Same pattern-matching idea as relation extraction
- Patterns are written over the question text itself
- **Example — the birth-year relation**
  - Look for the question word "When"
  - Look for a main verb such as "born"
  - Extract the named entity argument of that verb
- High precision, but coverage is limited to relations you hand-code

Supervised Methods

Question + logical-form pairs (training data) → Parse each question → Align parse tree to the logical form → Induce general rules

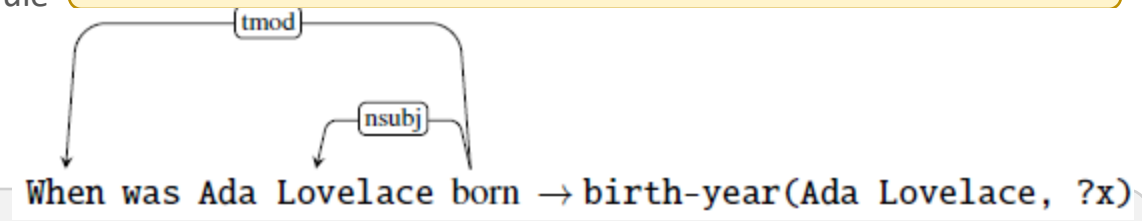
BOOTSTRAP: SEED RULES + LEXICON

Lexicon of entities: Texas, Ada Lovelace



WORKED EXAMPLE — ONE TRAINING TUPLE

Question	"When was Ada Lovelace born?"
Parsed (tmod, nsubj)	When was Ada Lovelace born
Logical form	birth-year (Ada Lovelace, ?x)
Induced rule	"When was · born" → birth-year(·, ?x)



Count how often each rule fires → normalise to probabilities → pick the highest-probability parse when a question has several readings

SCALING UP — BEYOND SINGLE RELATIONS

"What is the biggest state bordering Texas?" → argmax(λx.state(x) ∧ borders(x, texas), λx.size(x))

Needs argmax and word-to-relation mappings (biggest → size); richer default rules break each example into reusable tuples — GeoQuery (Zelle & Mooney, 1996); Zettlemoyer & Collins (2005)

IBM Watson: The DeepQA Architecture

Watson (Jeopardy! champion, 2011) answers by combining text and knowledge-base evidence — four stages

1 Question Processing

- Parse, NER, coreference, relations
- Extract focus + lexical answer type
- Classify question type

2 Candidate Generation

- Query triple stores (DBpedia, IMDB)
- Passage search over text
- Wikipedia titles as candidates

3 Candidate Scoring

- Answer-type match via WordNet/DBpedia
- Time & space consistency checks
- Supporting-passage overlap

4 Merging & Ranking

- Merge synonyms (JFK = John F. Kennedy)
- Classifier scores each feature vector
- Highest-confidence answer wins

Worked example

"He was a bank clerk in the Yukon before he published 'Songs of a Sourdough' in 1907."

Focus: he • Answer type: poet / clerk • Relation: authorOf(focus, "Songs of a Sourdough")

Why it scales

- ~5,000 answer types seen in 20,000 questions
- Types score evidence — they don't filter answers

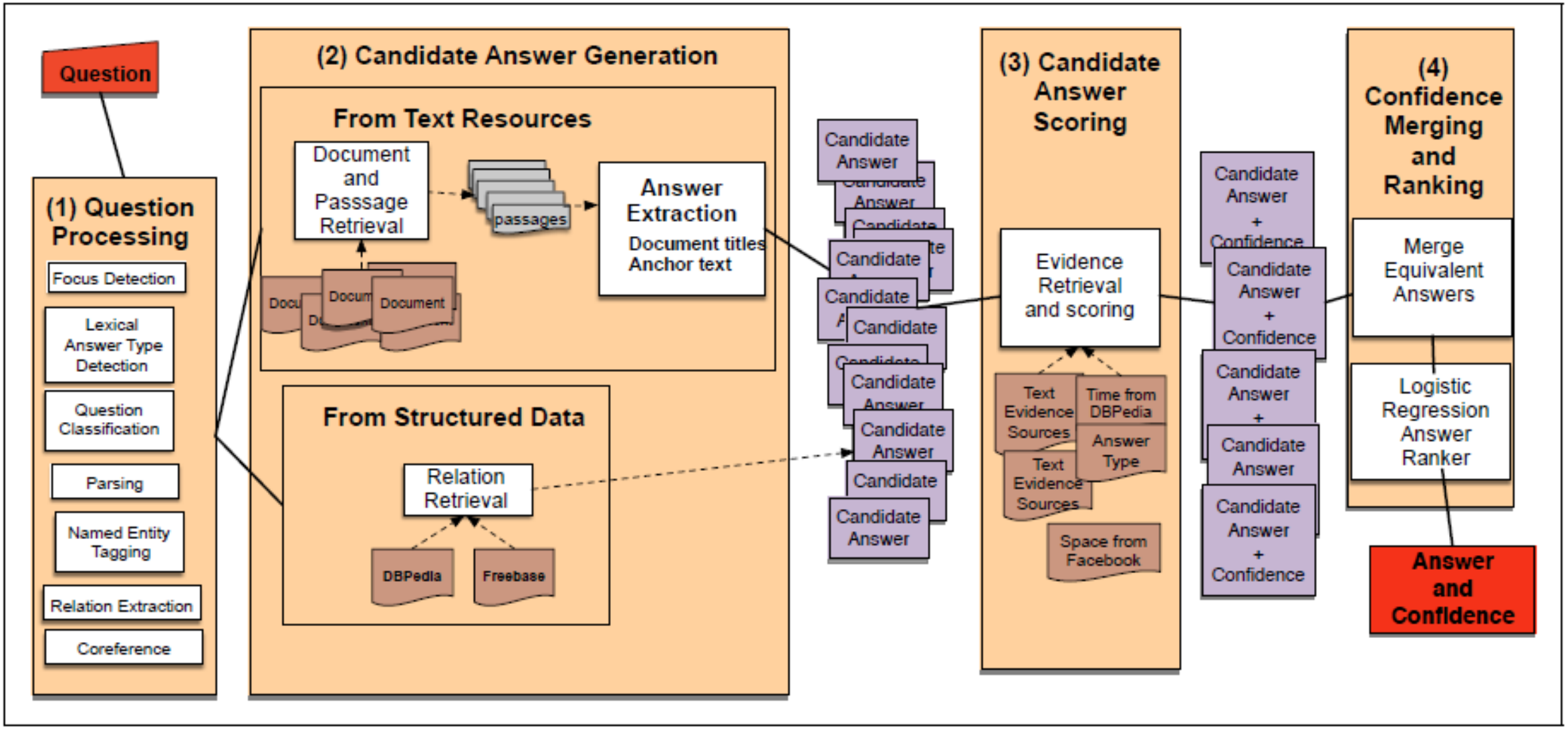


Figure 25.11 The 4 broad stages of Watson QA: (1) Question Processing, (2) Candidate Answer Generation, (3) Candidate Answer Scoring, and (4) Answer Merging and Confidence Scoring.

Evaluation of Factoid Answers

Two evaluation families: ranked factoid answers (MRR) and reading comprehension (Exact Match, F1)

Factoid QA — Mean Reciprocal Rank (MRR)

- Introduced in the TREC Q/A track, 1999
- Needs human-labelled answers + a ranked answer list
- Score per question = 1 / rank of first correct answer
- No correct answer in the set → score 0
- System score = average across all N questions

$$\text{MRR} = \frac{1}{N} \sum_{i=1 \text{ s.t. } rank_i \neq 0}^{N} \frac{1}{rank_i}$$

Reading Comprehension — SQuAD metrics

Both ignore punctuation and articles (a, an, the)

Exact Match

% of predictions identical to the gold answer

F1 Score

Token-overlap F1, averaged over all questions

Why two metrics

Exact Match is strict; F1 rewards partially correct spans

Question Answering Datasets

Benchmarks span factoid retrieval, knowledge-base parsing, and reading comprehension

Dataset	Size	Source / Answer form	What makes it distinctive
TREC QA (1999-2004)	Competition sets	Handwritten answers	Used by early factoid systems
TriviaQA (2017)	650K triples / 95K pairs	Wikipedia + web evidence	~6 supporting documents per question
WebQuestions (2013)	5,810 questions	Freebase answers	Web-user wh- questions, one entity each
WebQuestionsSP (2016)	Subset of the above	SPARQL semantic parses	Only questions answerable from Freebase
ComplexWebQuestions (2018)	34,689 questions	Answers, snippets, SPARQL	Compositional and complex questions
SQuAD / WikiQA	Large-scale	Spans in short passages	Standard reading-comprehension baselines
NarrativeQA (2018)	Long documents	Books and movie scripts	Questions require whole-document reading
QuAC (2018)	100K questions	Dialogue over hidden text	Two crowd workers ask and answer in context
MCTest (2013)	500 short stories	Multiple choice	Children's fiction written by crowd workers
ARC (2018)	Science questions	Multiple choice	Designed to defeat lexical co-occurrence cues

Dialog System

Textbook: Dialog Systems and Chatbots.pdf

Dialog Systems

- Conversational agents communicate with users in natural language - text, speech, or both
- Two broad classes
  - Task-oriented dialogue agents - use conversation to get a task done
  - Chatbots - designed for extended, unstructured conversation
- Where task-oriented agents show up
  - Digital assistants: Siri, Alexa, Google Home, Cortana
  - Directions, appliance control, restaurant search, calls
  - Corporate website Q&A and robot interfaces
  - Social good: DoNotPay, a “robot lawyer” for parking fines, housing, asylum claims
- Chatbots mimic human-human chat - mainly entertainment, but also make task agents feel natural

Properties of Human Conversation

- Human conversation is an intricate joint activity, not simple message exchange
- Understand how people converse before designing an agent
- Six subtle properties shape every dialogue
  - **Turns** - who speaks when, and how speakers hand over
  - **Speech acts** - what an utterance does, not just what it says
  - **Grounding** - confirming that what was said was understood
  - **Dialogue structure** - how utterances link into coherent exchanges
  - **Initiative** - who drives the conversation at each moment
  - **Implicature** - meaning inferred beyond the literal words
- These properties are why natural dialogue systems remain hard to build

Six Properties of Human Conversation

Six characteristics that make human dialogue hard to model - and hard for systems to imitate

1 Turns

- A dialogue is a sequence of turns: A1, B1, A2 ...
- A turn can be one word or several sentences
- Systems must know when to stop and when to start
- Endpointing: detecting when the user is done

2 Speech Acts

- Every utterance performs an action (Austin, 1962)
- Constatives - stating, claiming, denying
- Directives - asking, requesting, ordering
- Commissives – promising, planning
- Acknowledgments – thanking, greeting

3 Grounding

- Participants build common ground (Stalnaker, 1978)
- Grounding = signalling that you understood
- Say “OK”, repeat the words, or start with “And”
- Like an ACK, or an elevator button lighting up

4 Dialogue Structure

- Adjacency pairs: question → answer, proposal → accept / reject
- Side sequences interrupt: corrections, clarifications
- Presequences check capability before requesting
- “Can you make train reservations?” → then the request

5 Initiative

- Who drives the conversation at a given moment
- User-initiative: search engines, QA systems
- System-initiative: rigid, often frustrating
- Mixed initiative is human-normal but hard to build

6 Implicature

- Speakers mean more than the words say (Grice, 1975)
- Guided by maxims - above all, relevance
- “I need to be there for a meeting on the 12th”
- → infer the flight should be on the 11th

GUS: Frame-based Dialogue Systems

Task-based dialogue: help the user book a flight or buy a product - GUS (1977) still underlies today's assistants

Task-oriented = What does the system want to accomplish?
Frame-based = How does the system organize the information needed to accomplish it?

The GUS architecture

- Goal-driven: complete a concrete user task
- Introduced for travel planning (Bobrow et al., 1977)
- Simple, but astonishingly long-lived
- Basis of most commercial digital assistants

Slot types in the hotel booking system:

```
Hotel Booking Frame
-------------------
City       → Bengaluru
Check-in   → 20 August
Check-out  → 22 August
Guests     → 2
Room Type  → ?
```

The system asks questions to **fill the missing slots and progressively fills the frame**.

Everything is built on frames

- Frame = the intentions the system can extract
- Each frame holds a collection of slots
- Every slot takes values of one semantic type
- All the frames together = the domain ontology

Example: the DATE frame

MONTH: NAME YEAR: INTEGER

DAY: BOUNDED-INTEGER 1-31

WEEKDAY: MEMBER (Sunday ... Saturday)

Control structure for frame-based dialogue

- Frame-based control powers assistants - Siri, Alexa, and Google Assistant
- Goal: fill the frame's slots with the user's intended values, then act
- The system asks template questions per slot; one reply can fill several slots
- Condition-action rules on slots set defaults across related frames
- Multiple frames per domain, so the system disambiguates and switches control
- GUS is a production rule system: inputs fire productions that switch control
- With enough information, it queries the database and returns the result

One response can fill multiple slots

User: "I want a flight from San Francisco to Denver one way leaving after five p.m. on Tuesday."

FLIGHT-BOOKING frame — slots filled from one response

- **ORIGIN-CITY:** San Francisco
- **DEST-CITY:** Denver
- **TRIP-TYPE:** one way
- **DEPART-TIME:** after 5 p.m.
- **DEPART-DAY:** Tuesday
- **AIRLINE:** still to be asked
- **SEAT-CLASS:** still to be asked

Filled slots are skipped; the system asks only for the grey ones

Slot	Type	Question Template
ORIGIN CITY	city	"From what city are you leaving?"
DESTINATION CITY	city	"Where are you going?"
DEPARTURE TIME	time	"When would you like to leave?"
DEPARTURE DATE	date	"What day would you like to leave?"
ARRIVAL TIME	time	"When do you want to arrive?"
ARRIVAL DATE	date	"What day would you like to arrive?"

Figure 26.9 A frame in a frame-based dialogue system, showing the type of each slot and a question used to fill the slot.

Filling a medical consultation frame

System: What is your main problem?

Patient: I have had a headache for three days.

Chief complaint → headache | Duration → three days

System: Do you have fever?

Patient: Yes, since yesterday.

Fever → yes | Since → yesterday

Slot	Value
Patient name	?
Age	?
Chief complaint	headache
Duration	three days
Temperature	?
Blood pressure	?
Symptoms	fever since yesterday
Medication	?

Each turn fills more of the frame; unfilled slots stay marked ?

Natural language understanding for filling slots in GUS

- NLU in a frame-based system extracts three things from the user's utterance
- **Domain classification:** is the user talking about airlines, an alarm clock, or the calendar?
  - This 1-of-n choice is unnecessary for single-domain systems, but multi-domain assistants are the modern standard
- **Intent determination:** the user's general task or goal — Find a Movie, Show a Flight, Remove a Calendar Appointment
- **Slot filling:** extract the particular slots and fillers the user intends, given that intent

From utterance to filled frame

"Show me morning flights from Boston to San Francisco on Tuesday"

DOMAIN: AIR-TRAVEL
INTENT: SHOW-FLIGHTS
ORIGIN-CITY: Boston
DEST-CITY: San Francisco
ORIGIN-DATE: Tuesday
ORIGIN-TIME: morning

"Wake me tomorrow at 6"

DOMAIN: ALARM-CLOCK
INTENT: SET-ALARM
TIME: 2017-07-01 0600-0800

GUS slot filling: handwritten rules in condition-action rules on slots

```
SET-ALARM  →  wake me (up) | set (the|an) alarm | get me up
```

The Dialogue-State Architecture

- Modern task-based systems use the dialogue-state (belief-state) architecture, a richer version of the frame-based design
- **NLU component**
  - Extracts slot fillers from the utterance, using machine learning rather than hand-written rules
- **Dialogue state tracker**
  - Maintains the current state: the user's latest dialogue act plus all slot-filler constraints so far
  - Hard because ASR and understanding errors are common; crucial because the policy depends on it
- **Dialogue policy**
  - Decides what the system does or says next
  - GUS simply asked until the frame was full; richer policies choose when to answer, clarify, or suggest
- **Natural language generation**
  - GUS used pre-written templates; modern generators condition on context for more natural turns

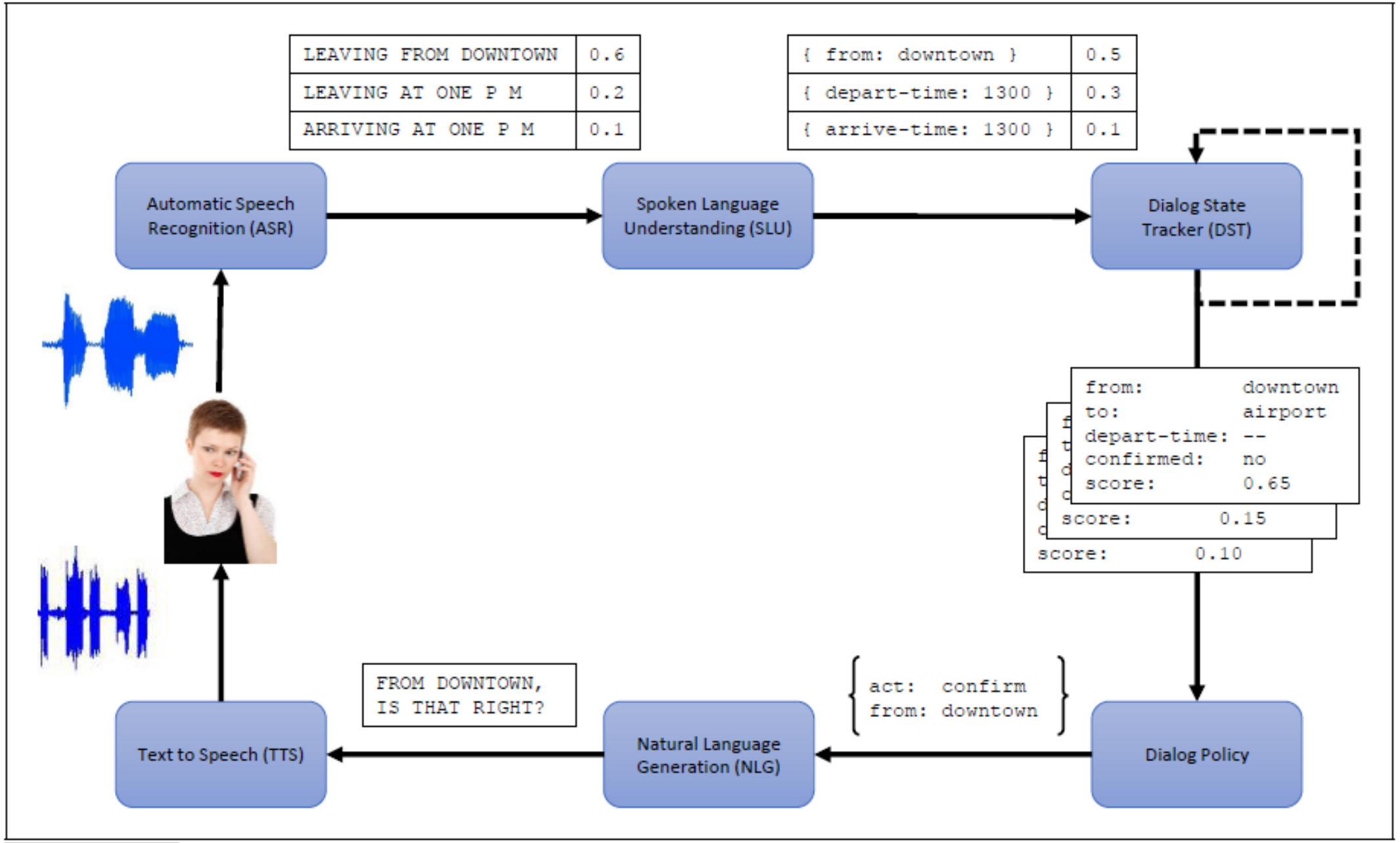


Figure 26.11 Architecture of a dialogue-state system for task-oriented dialogue from Williams et al. (2016).

Dialog System Design

- Users matter more here than in most speech work, so design ties closely to Human-Computer Interaction
- Designing strategies, prompts, and error messages is **voice user interface design**, following **user-centered principles**
- **Study the user and task**: interviews, similar systems, and human-human dialogues
- **Prototype with Wizard-of-Oz**: users think they address software, but a hidden human responds
- It tests the architecture early and yields training data, but cannot mimic real errors or timing
- **Test iteratively**: user testing showed people barge in, prompting support for overlapped speech

Ethical Issues in Dialogue System Design

- Ethics in artificial agents predates conversational AI — a concern at the heart of Frankenstein
- Bias: learning systems replicate the biases in their training data, and chatbots approximate those responses
- Microsoft's Tay (2016) was pulled after 16 hours for slurs and attacks learned from users
- Henderson et al. (2017) found hate speech and bias in Twitter, Reddit, and movie corpora — and in chatbots trained on them
- Privacy: home agents may record secrets, which a trained model can leak to an adversary who knows the keyphrase
- Transcripts must be anonymized, with Institutional Review Board oversight
- Gender equality: chatbots are mostly given female names and evade rather than reject harassment

https://medium.com/startup-grind/google-home-vs-alexa-56e26f69ac77